CS110 / INDEPENDENCE ROAD

prep / Nicolas Uribe

Issued: Nov 10, 2026, 2:35 PM Pacific

Record: PREP-D4A28E0760DF9C2EC199B0C9

Personal public verification code: VIEW-57A8237D0C4986D619F76DCD

Include your own public code with every assignment, even in a team folder. Never submit your private recovery key.

SHA-256: dc47e09aad3d7a3b8f2162ac767ce37a083c27e97463ae50fc0990ce61e7879d

Verify the record number at courses.addp2p.com/development-road/independence/index.html?view=verify

An issued record preserves submitted information. It is not a grade or proof of comprehension.

Personal GitHub repository URL

https://github.com/nicolasuribe0716-ui/12A-Independence-Road-assignment-

Git test: edit run/preview save commit push open remote. Record the test commit and result.

I tested my Git workflow by making an edit to my project, running it to preview the changes, and saving my work. I then committed the changes with a descriptive commit message and pushed them to my GitHub repository. Finally, I opened the remote repository to verify that my latest changes were successfully uploaded.

Instructor access and team Git readiness: what did you test? Solo: say solo.

I worked solo on this assignment. I opened my CS110 project in Visual Studio and checked that I could access my project files. I am working independently, so I did not need to test team collaboration. My next step is to test my Git workflow by making a small edit, saving it, committing the changes, and pushing them to GitHub to verify that everything works correctly.

1 Introduction scan notes, one useful idea and one question

Observation / Summary:

Scanned Malik s introductory game development pipeline video, which outlines a four-step framework covering developer mindset, project folder architecture, living documentation, and tool/IDE setup.

Useful Idea:

Establishing a clean folder structure and treating documentation as a living process from day one prevents asset clutter and keeps projects scalable.

Question:

What custom folder structure utility or template will best fit our team's specific game engine workflow?

2 Developer mindset scan notes, one useful idea and one question

Scanned Malik s mindset module for game development, which outlines strategies for building a growth mindset, navigating core project obstacles (technical bugs, scope creep, funding, and burnout), and maintaining momentum across long production cycles.

Useful Idea:

Scope creep and burnout are best managed by breaking large milestones into small, trackable task lists and protecting developer well-being with regular breaks and strict goal boundaries.

Question:

CS110 / INDEPENDENCE ROAD

How can we best set up clear communication channels and task boundaries to prevent scope creep before starting execution?

3 Folders and project creation scan notes, one useful idea and one question

Observation / Summary:

Walked through Malik s folder and file creation pipeline tutorial, which compares manual setup against automated JSON-driven project architecture generation for game development.

Useful Idea:

Automating directory setup using a template JSON tool and setting up automated timestamped project logs saves significant time and ensures clean, standardized onboarding for any project.

Question:

What specific custom folders or asset subcategories should we add to our project's JSON template for our unique game style?

4 Documentation scan notes, one useful idea and one question

Observation / Summary:

Explored Malik s documentation pipeline tutorial, focusing on how to structure a game design document (GDD) and maintain it as a living reference throughout production.

Useful Idea:

Writing down the elevator pitch, core loop, and system boundaries in a centralized GDD early on prevents scope creep and costly reworks.

Question:

How should we organize our project's GDD template so that both design and technical notes remain easily scannable for everyone?

5 Introduction to Unity scan notes, one useful idea and one question

Observation / Summary:

Walked through Malik s Unity introduction tutorial, focusing on how to install and configure the Unity game engine and link it to our pre-established project folder architecture.

Useful Idea:

Setting up the engine workspace and matching its directory references directly to your structured folder templates ensures smooth asset loading from day one.

Question:

What specific Unity version and package plugins should we install to match our project's core requirements?

6 Import a custom player scan notes, one useful idea and one question

Observation / Summary:

Walked through Malik s asset import tutorial, focusing on how to bring a custom 3D player character into Unity and configure its import settings and folder mappings.

Useful Idea:

Keeping textures and models properly sorted in your project folders before importing prevents broken material links and missing texture paths in the engine.

Question:

What specific import scale and material settings should we configure to ensure our custom character matches the game world's proportions?

Your interpretation: what can this pipeline contribute to a prior or new story?

Observation / Summary:

Connected the foundational game development pipeline framework spanning mindset, automated folder structures, GDD documentation, and engine import workflows to overall project execution.

Useful Idea:

Integrating structured planning tools, standardized file architectures, and continuous documentation practices early on transforms abstract creative visions into an organized, scalable production pipeline.

Question:

How can we best adapt this standardized pipeline to fit our team's upcoming collaborative design sprint?

Questions to resolve before committing to a team or mission

tform Response Entry

Observation / Summary:

Reviewed the core pipeline requirements, covering mindset management, automated directory structures, living documentation (GDDs), and Unity asset importation.

Useful Idea:

Resolving structural, technical, and collaborative questions upfront ensures everyone aligns on scope and execution before committing to a development team or mission.

Question:

Do all team members have their development environments, JSON folder templates, and version control workflows fully aligned and tested before we start production?

Whiteboard

The game development mini-pipeline is structured around four interconnected sequential phases that scale from conceptual preparation to engine execution:

Mindset & Strategy: Fostering growth mindsets, overcoming common technical/scope hurdles, and maintaining motivation.

Folder Structure & Architecture: Organizing root directories, project logs, and automated JSON directory generation.

CS110 / INDEPENDENCE ROAD

Documentation: Establishing living Game Design Documents (GDDs), core loops, and task lists to prevent scope creep.

Engine Implementation & Assets: Importing custom player models, configuring Unity, and setting up workspace workflows.

Useful Connection / Interpretation:

Connecting each phase demonstrates that technical execution in Unity relies entirely on upfront architectural planning, structured documentation, and disciplined project management.

